Whatever you change this to, you need to configure the size of the Active Record connection pool to match the number of threads.[23]

The second option is to add more server processes. These will consume more CPU and memory than adding threads, but it is often easier to modify the number of processes than it is to modify the number of threads. How you do this depends on how you run your app in production. The Sidekiq server process is run via `bundle exec sidekiq`. Often, running more server processes means allocating more virtual servers to run that process, but it depends on how you have it all set up. Some platforms allow you to scale up and down by adding or removing processes.

The last option is to combine all the techniques we've learned in this section, which is to run multiple Sidekiq servers with individualized configurations. For example, you could run `bundle exec sidekiq`, which will behave as discussed: one process with five threads, processing the high queue first, then the default queue. You could additionally run a command like `bundle exec sidekiq -q one_at_a_time -c 1`, which would run a new process that only processes the queue "one_at_a_time" with only one thread, effectively processing jobs in that queue in a serialized way. `bundle exec sidekiq -h` will show you additional options. The command line options override whatever is in `config/sidekiq.yml`.

The proper way to leverage these options highly depends on your app and workload. Observe how your app behaves. Measure it and make changes only to address a specific problem. If your changes don't fix it, roll them back and try something else.

One last thing to talk about is how to manage the code in the jobs themselves. We've already seen some duplication across jobs, and we still have `ApplicationJob` hanging around, configured to use Active Job, which we haven't discussed.